

LIVE-SCIENCE

A Verifiable Dynamic Content Generation System for Science Education

Project Technical Report

Based on the Current Repository Architecture and Production Pipeline

Based on the current main branch

6 August 2026

Purpose. This report describes the architecture, production workflow, technical principles, hardware strategy, and demonstrated results of Live-Science. It focuses on how the system turns scientific explanations into verifiable dynamic educational content.

Abstract.

Live-Science converts written scientific material into programmatic and model-generated educational media. Its central design separates a deterministic Phase 1 teaching video from optional realistic keyframe generation in Phase 2 and continuous video generation in Phase 3. This allows precise relationships to remain programmatic while selected scenes receive realistic treatment through FLUX.2 and LTX-2.3. A React and FastAPI product layer supports content submission, generation jobs, progress review, and artifact delivery. The design emphasizes traceability, staged validation, and controlled AMD Radeon GPU use under ROCm.

Keywords: science education, programmatic animation, generative video, deterministic rendering, Bridge manifest, AMD Radeon, ROCm, FLUX.2, LTX-2.3.

1. Introduction

Image and video generation models can turn written information into visual media, but scientific education requires more than visual quality. A useful teaching video must preserve source meaning, show relationships at the correct time, maintain labels and state transitions, and leave enough evidence for a reviewer to understand how the result was produced. A visually attractive but semantically incorrect sequence is not a successful output.

Live-Science treats media generation as an auditable production process. Phase 1 creates a deterministic program, subtitles, rendered review artifacts, and a Bridge declaration of route, key moments, events, and continuity requirements. Content that is sufficiently explained by the program video can stop there. Content that benefits from realistic appearance can continue through controlled image and video generation, with intermediate files retained at phase boundaries.

1.1 Project Objectives

1. Reduce the effort required to convert textbooks, lecture notes, articles, and explanatory scripts into dynamic educational content.
2. Preserve scientific meaning through explicit phase contracts, deterministic time-state definitions, and reviewable intermediate artifacts.
3. Use generative models selectively, with stable references and recorded parameters, instead of treating generation as an uncontrolled final step.
4. Support reproducible execution and diagnosis on AMD Radeon GPU systems through ROCm-aware operating policies.

2. Current Repository Architecture

The repository has two main parts. The `modules/` directory contains the three-phase media-production workflow, while `web/` contains the user-facing application. The production phases are organized directly as Phase 1, Phase 2, and Phase 3, with shared prompts, policies, tools, and run artifacts.

2.1 Complete Production Workflow

The production layer organizes work into three explicit stages. Phase 1 produces a programmatic instructional video and the Bridge information needed by later phases. Phase 2 selects moments that benefit from realistic representation and creates visual anchors. Phase 3 generates transitions between those anchors and assembles the final educational video. Each stage writes reviewable artifacts so that the production process can be inspected and repeated.

2.2 Web Product Layer

The product layer combines a React and Vite frontend with a FastAPI backend. Users can submit teaching material, choose a generation approach, monitor progress, and inspect or download the resulting media. The backend manages asynchronous work, stores job state, reports system health, and connects the interface to the production tools.

2.3 Product and Production Integration

The Web application reuses the same Phase 1 rendering, Bridge, realistic-image, and video-generation components as the production workflow. This creates a direct path from educational text to a programmatic or realistic result. Further work will continue to bring the product experience and the full production review process closer together.

3. Core Algorithms and Data Contracts

3.1 Deterministic Time-State Rendering

Phase 1 scenes expose a deterministic time interface that can be summarized as

$$\text{frame} = \text{renderFrame}(\text{state}, t).$$

Key moments, subtitles, posters, and exported frames refer to the same declared timeline. Given the same program source, parameters, environment, and time value, a reviewer can reproduce a frame and relate visible state to the corresponding implementation.

3.2 Bridge Contract

The Bridge manifest is the formal handoff from Phase 1. Its current contract can record media metadata, route, target style, world-continuity requirements, key moments, events, and references to presentation, clean, and overlay assets. Subtitles, rendered videos, reports, and validation results belong to the surrounding run directory; they should not all be described as fields inside the manifest itself. Downstream phases must preserve the Phase 1 event order and object identity rather than independently reinterpreting the teaching process.

3.3 Anchor Selection and Controlled Image Generation

For scenes that benefit from realistic representation, Phase 2 uses structural frames, scene descriptions, preservation constraints, and world continuity to produce a small set of realistic anchors. FLUX.2/dev provides the image-generation path, while the retained prompts and outputs support later review.

3.4 Segment-Based Video Generation

Phase 3 uses approved anchors as first- and last-frame constraints for LTX-2.3 segments. Segmenting the process limits the cost of a failure and makes temporal or topological defects easier to isolate. The complete production path can introduce a derived intermediate anchor when one transition asks the model to perform too many changes at once. Final delivery may then remap overlays and subtitles onto the generated base video and validate the assembled artifact.

3.5 Validation Scope

Production validation distinguishes executable output from deliverable educational content. Automated checks inspect the expected media and review artifacts, while visual review considers semantic correctness, subtitle placement, continuity, and physical plausibility.

4. Models and Production Components

4.1 Language Model Component

A language-model component supports interpretation, script organization, route selection, and program preparation. The architecture is defined by the structured artifacts passed between phases rather than by one permanent language-model version.

4.2 FLUX.2/dev and LTX-2.3

Phase 2 uses FLUX.2/dev for realistic anchor generation, and Phase 3 uses LTX-2.3 to create constrained continuous segments. Editing, annotation, subtitle placement, assembly, and delivery validation remain deterministic production steps outside the generative models.

5. AMD Radeon GPU and ROCm Adaptation

Image and video inference can consume substantial device memory, so the production policy prioritizes reliable execution over maximum concurrency. Inputs and generation settings are checked before expensive work, and low-cost smoke runs are used to expose configuration or memory problems before full production.

5.1 Serialization and Resource Control

High-memory generation tasks run serially so that image and video models do not compete for the same device memory. CPU-side planning, validation, and inspection can continue when they do not interfere with model execution.

5.2 Failure Handling and Observability

Generation records preserve the information needed to understand a result or diagnose a failure. The product interface exposes job progress, errors, artifacts, and model availability, while future monitoring can make GPU behavior easier for operators to understand.

6. Demonstrated Case: Oxbow-Lake Formation

6.1 Educational Objective and Phase 1

The oxbow-lake demonstration explains how lateral erosion and deposition enlarge a river meander, narrow its neck, create a flood cutoff, and eventually isolate a crescent-shaped lake. Phase 1 presents this process from a stable overhead view and establishes the instructional order, visual continuity, and key changes that later phases must preserve.

6.2 Phase 2 Anchors

Phase 2 converts selected stages of the river's development into realistic keyframes. A shared world reference keeps the viewpoint, floodplain, river appearance, lighting, and geographic scale consistent while preserving the difference between the active channel and the abandoned meander.

6.3 Phase 3 Segments and Assembly

Phase 3 generates short transitions covering meander growth, neck narrowing, cutoff formation, gradual isolation, and final oxbow-lake separation. An additional intermediate state helps the model express sediment accumulation as a gradual process rather than an abrupt visual jump. The generated segments are then assembled with teaching overlays, stage labels, and subtitles to create the final English educational video.

6.4 Interpretation and Limitations

The case demonstrates the intended separation of responsibilities. Phase 1 fixes the scientific sequence, Phase 2 establishes a coherent realistic world, and Phase 3 creates continuous motion before deterministic delivery work is applied. Some subtle riverbank changes remain challenging for generative video, so the result should be understood as a validated workflow demonstration rather than a claim of perfect physical simulation.

7. Current Strengths

1. Programmatic and generative media are complementary routes selected according to teaching requirements rather than treated as interchangeable quality levels.
2. The Bridge and run directories provide explicit, inspectable handoffs between semantic intent, rendering, realistic anchors, continuous generation, and delivery.
3. Prompts, manifests, reports, and media metadata make the demonstrated production process auditable.
4. The production policies explicitly address VRAM safety, serial execution, smoke testing, bounded retries, and retained failure evidence.

8. Limitations and Future Work

8.1 Product Integration

The product interface and production workflow already share important components, but their end-to-end review experience can be made more consistent. Future work should make phase transitions, approvals, recovery, and final validation equally clear whether a run begins from the Web interface or from the production workflow.

8.2 End-to-End Product Parity

The next engineering priority is to strengthen the connection between the product experience and the complete production workflow. This includes clearer approval checkpoints, reliable recovery, consistent composition, and final validation across all generation routes.

8.3 Scientific and Educational Evaluation

Programmatic scenes remain simplified representations and require domain review. Realistic generated media cannot be assumed to be scientifically correct because it looks plausible. Future checks should cover formulas, units, subtitle safe areas, object identity, topology, and domain-specific physical constraints. Broader studies should measure learner comprehension, authoring time, review effort, regeneration rate, and failure causes.

8.4 Hardware Monitoring

The product interface should expose useful information about GPU availability, stage duration, and failure reasons. This would make hardware behavior easier for operators to understand and support reliable execution across different Radeon devices.

9. Conclusion

Live-Science implements a practical three-phase architecture for converting scientific text into dynamic educational media. The oxbow-lake demonstration shows how a verified programmatic explanation can guide realistic keyframes, continuous generated motion, and deterministic delivery. The React and FastAPI layer provides an accessible product entry point and reuses the core production capabilities. Stronger review integration, scientific validation, hardware observability, and educational evaluation are the most important next steps.

References.

- [1] Live-Science repository overview and current project documentation.
- [2] Three-phase production workflow and Bridge specifications.
- [3] AMD Radeon GPU generation and review policy.
- [4] Live-Science Web product architecture and deployment documentation.
- [5] Oxbow Lake Formation demonstration artifacts and phase reports.